CSA0618-DAA

S.Harshavardhan

192524214

16. Test Different Types of Lists

```
#include <stdio.h>

void display(int arr[], int n)
{
    printf("[");
    for (int i = 0; i < n; i++)
    {
        printf("%d", arr[i]);
        if (i < n - 1)
            printf(", ");
    }
    printf("]\n");
}

int main()
{
    int a[] = {};
    int b[] = {1};
    int c[] = {7, 7, 7, 7};
    int d[] = {-5, -1, -3, -2, -4};

    printf("Empty list: []\n");

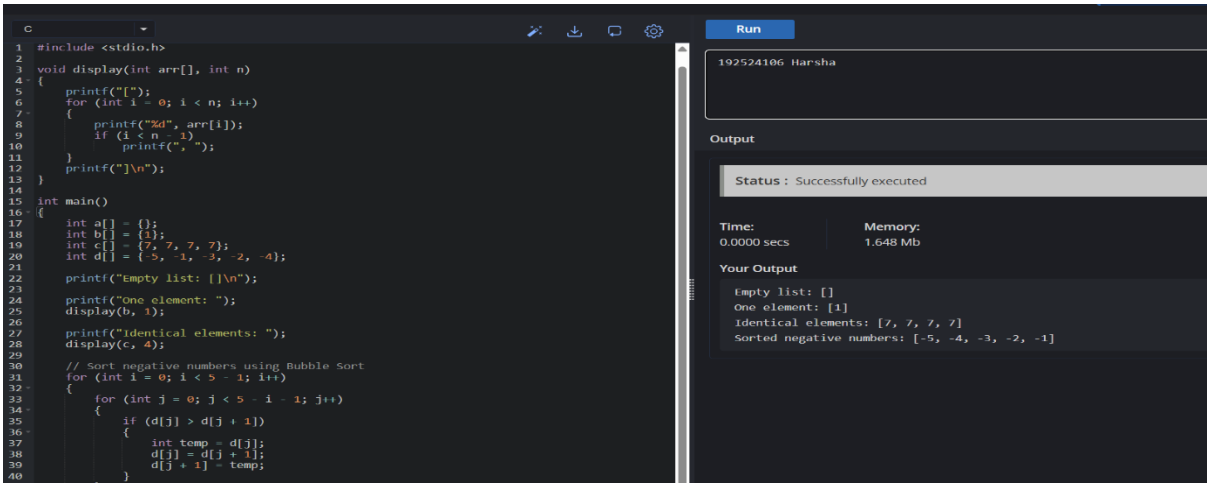
    printf("One element: ");
    display(b, 1);

    printf("Identical elements: ");
    display(c, 4);
```

```
    // Sort negative numbers using Bubble Sort
    for (int i = 0; i < 5 - 1; i++)
    {
        for (int j = 0; j < 5 - i - 1; j++)
        {
            if (d[j] > d[j + 1])
            {
                int temp = d[j];
                d[j] = d[j + 1];
                d[j + 1] = temp;
            }
        }
    }

    printf("Sorted negative numbers: ");
    display(d, 5);

    return 0;
}
```



17. Selection Sort

```
#include <stdio.h>
```

```
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int min = i;

        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
                min = j;
        }

        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
}

void display(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");
}

int main()
{
    int a[] = {5, 2, 9, 1, 5, 6};
    int b[] = {10, 8, 6, 4, 2};
    int c[] = {1, 2, 3, 4, 5};

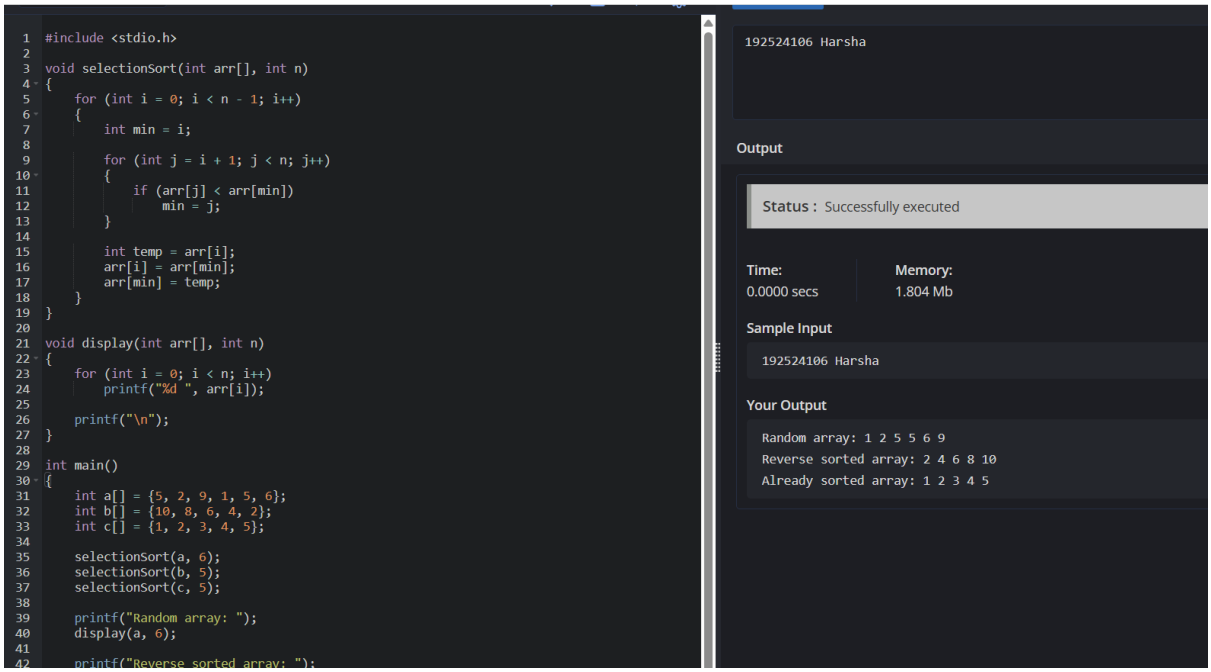
    selectionSort(a, 6);
    selectionSort(b, 5);
    selectionSort(c, 5);
```

```
    printf("Random array: ");
    display(a, 6);

    printf("Reverse sorted array: ");
    display(b, 5);

    printf("Already sorted array: ");
    display(c, 5);

    return 0;
}
```



19. Optimized Bubble Sort

```
#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int swapped = 0;

        for (int j = 0; j < n - i - 1; j++)
```

```
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }

        if (swapped == 0)
            break;
    }
}

int main()
{
    int a[] = {64, 25, 12, 22, 11};
    int b[] = {29, 10, 14, 37, 13};
    int c[] = {3, 5, 2, 1, 4};
    int d[] = {1, 2, 3, 4, 5};
    int e[] = {5, 4, 3, 2, 1};

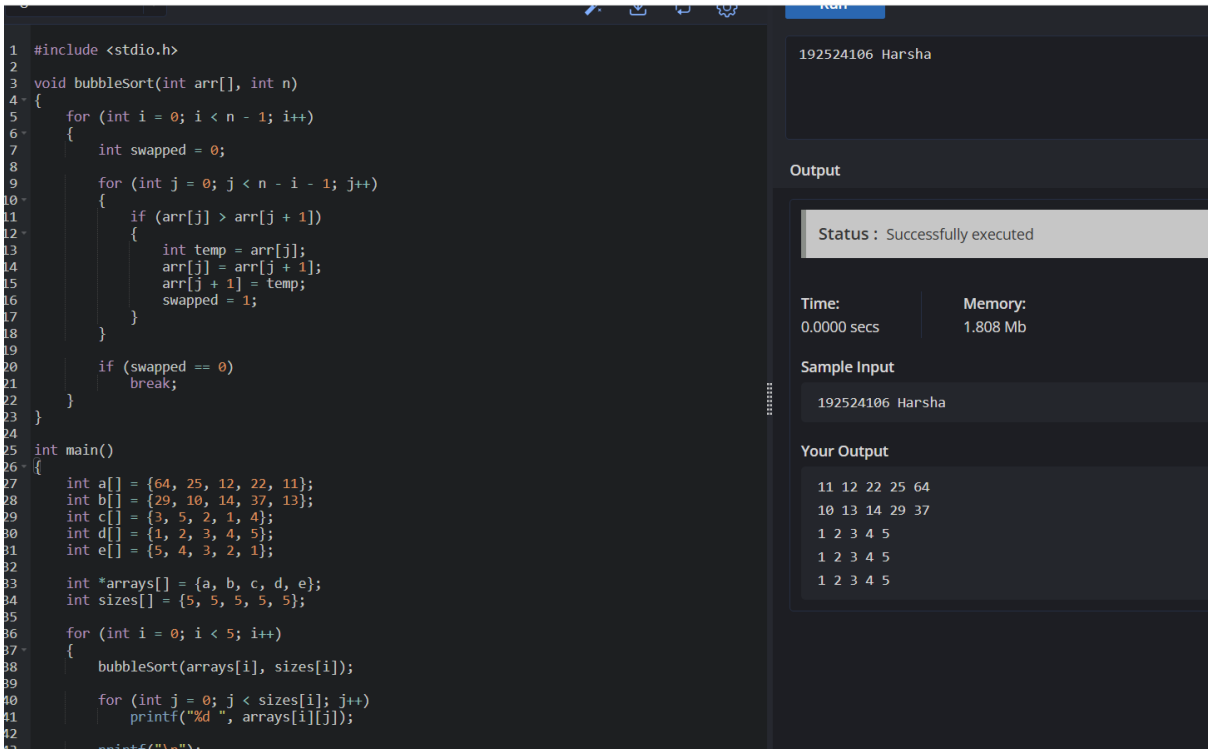
    int *arrays[] = {a, b, c, d, e};
    int sizes[] = {5, 5, 5, 5, 5};

    for (int i = 0; i < 5; i++)
    {
        bubbleSort(arrays[i], sizes[i]);

        for (int j = 0; j < sizes[i]; j++)
            printf("%d ", arrays[i][j]);

        printf("\n");
    }
```

```
    return 0;
}
```



```c
#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int swapped = 0;

        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }

        if (swapped == 0)
            break;
    }
}

int main()
{
    int a[] = {64, 25, 12, 22, 11};
    int b[] = {29, 10, 14, 37, 13};
    int c[] = {3, 5, 2, 1, 4};
    int d[] = {1, 2, 3, 4, 5};
    int e[] = {5, 4, 3, 2, 1};

    int *arrays[] = {a, b, c, d, e};
    int sizes[] = {5, 5, 5, 5, 5};

    for (int i = 0; i < 5; i++)
    {
        bubbleSort(arrays[i], sizes[i]);

        for (int j = 0; j < sizes[i]; j++)
            printf("%d ", arrays[i][j]);
```

```
192524106 Harsha

Output

Status : Successfully executed

Time:
0.0000 secs

Memory:
1.808 Mb

Sample Input

192524106 Harsha

Your Output

11 12 22 25 64
10 13 14 29 37
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

20. Insertion Sort with Duplicate Elements

```
#include <stdio.h>


void insertionSort(int arr[], int n)

{

   for (int i = 1; i < n; i++)

   {

      int key = arr[i];

      int j = i - 1;


      while (j >= 0 && arr[j] > key)

      {

         arr[j + 1] = arr[j];

         j--;

      }


      arr[j + 1] = key;

   }
```

```
}

void display(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

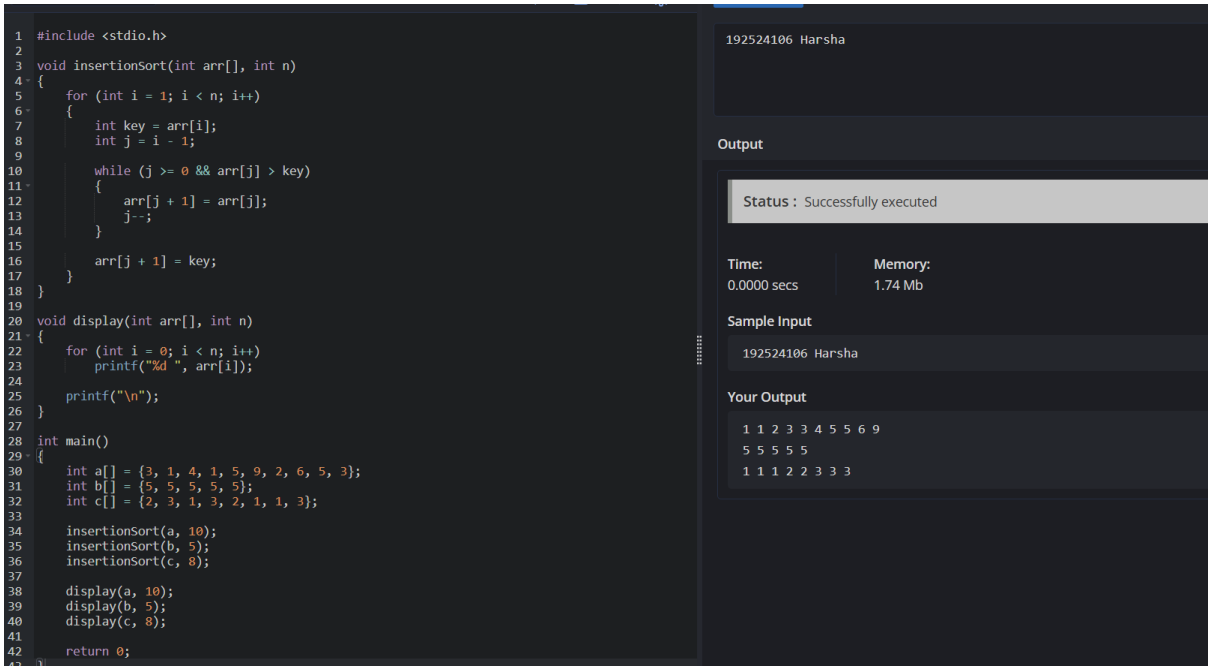
    printf("\n");
}

int main()
{
    int a[] = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};
    int b[] = {5, 5, 5, 5, 5};
    int c[] = {2, 3, 1, 3, 2, 1, 1, 3};

    insertionSort(a, 10);
    insertionSort(b, 5);
    insertionSort(c, 8);

    display(a, 10);
    display(b, 5);
    display(c, 8);

    return 0;
}
```



23. Kth Missing Positive Integer

```
#include <stdio.h>


int findKthPositive(int arr[], int n, int k)
{
    int missing = 0;
    int num = 1;
    int i = 0;

    while (missing < k)
    {
        if (i < n && arr[i] == num)
            i++;
        else
            missing++;
```

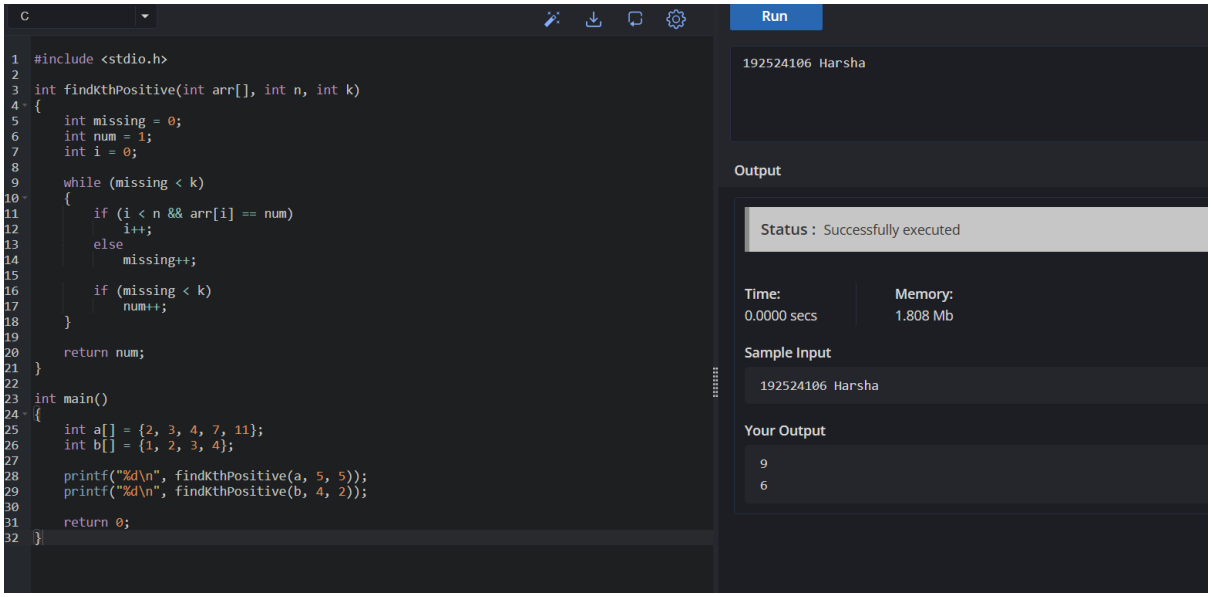
```
        if (missing < k)
            num++;
    }

    return num;
}

int main()
{
    int a[] = {2, 3, 4, 7, 11};
    int b[] = {1, 2, 3, 4};

    printf("%d\n", findKthPositive(a, 5, 5));
    printf("%d\n", findKthPositive(b, 4, 2));

    return 0;
}
```



25. Find a Peak Element

```
#include <stdio.h>

int findPeakElement(int nums[], int n)
```

```
{
    int left = 0;
    int right = n - 1;

    while (left < right)
    {
        int mid = left + (right - left) / 2;

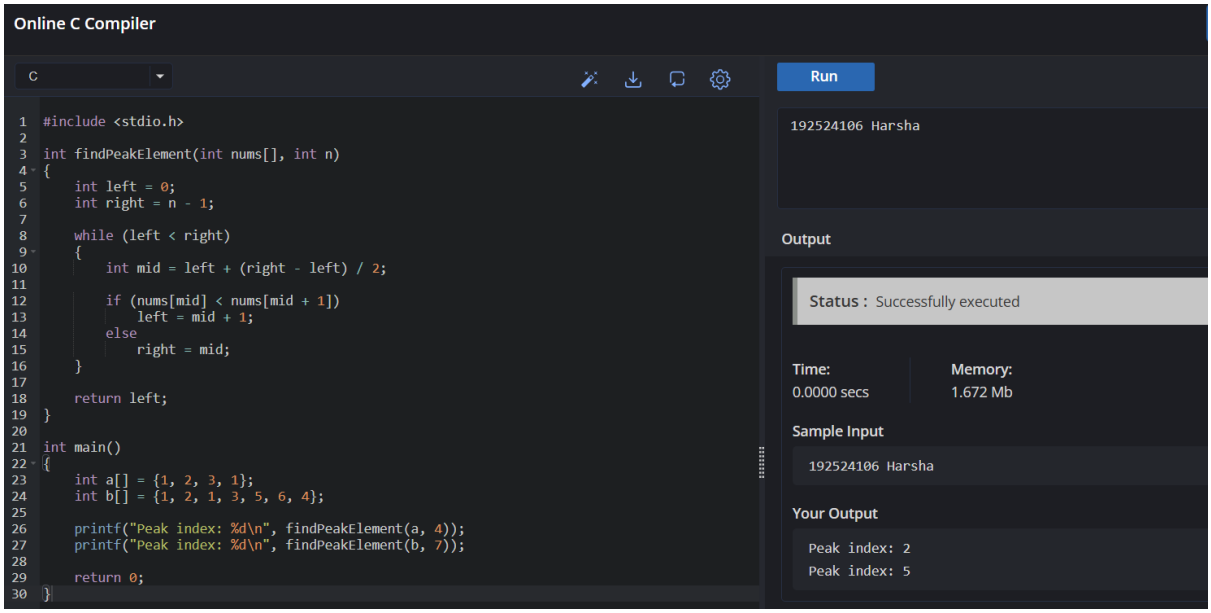
        if (nums[mid] < nums[mid + 1])
            left = mid + 1;
        else
            right = mid;
    }

    return left;
}

int main()
{
    int a[] = {1, 2, 3, 1};
    int b[] = {1, 2, 1, 3, 5, 6, 4};

    printf("Peak index: %d\n", findPeakElement(a, 4));
    printf("Peak index: %d\n", findPeakElement(b, 7));

    return 0;
}
```



26. Convex Hull Using Brute Force

```
#include <stdio.h>

#include <string.h>


int strStr(char haystack[], char needle[])

{

   int n = strlen(haystack);

   int m = strlen(needle);


   if (m == 0)

      return 0;


   for (int i = 0; i <= n - m; i++)

   {

      int j;


      for (j = 0; j < m; j++)

      {

         if (haystack[i + j] != needle[j])

            break;

      }


      if (j == m)
```

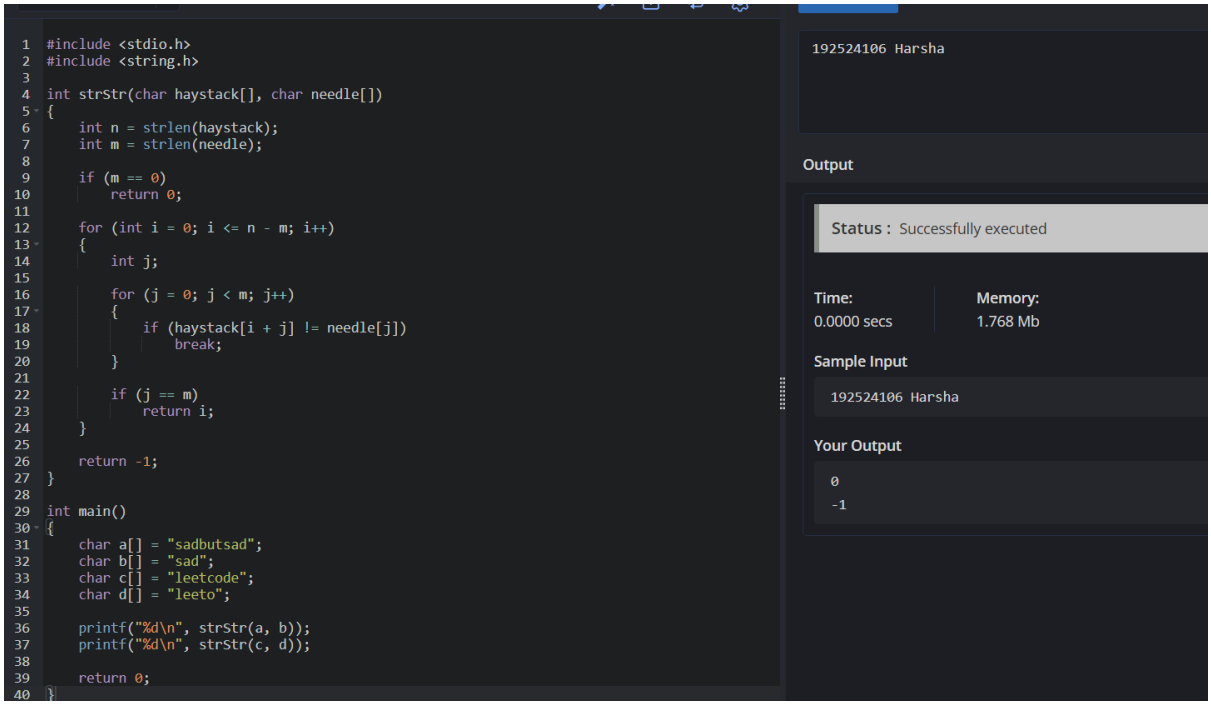
```
            return i;
        }

    return -1;
}

int main()
{
    char a[] = "sadbutsad";
    char b[] = "sad";
    char c[] = "leetcode";
    char d[] = "leeto";

    printf("%d\n", strStr(a, b));
    printf("%d\n", strStr(c, d));

    return 0;
}
```



27. Convex Hull in Counter-Clockwise Order

```
#include <stdio.h>
#include <string.h>

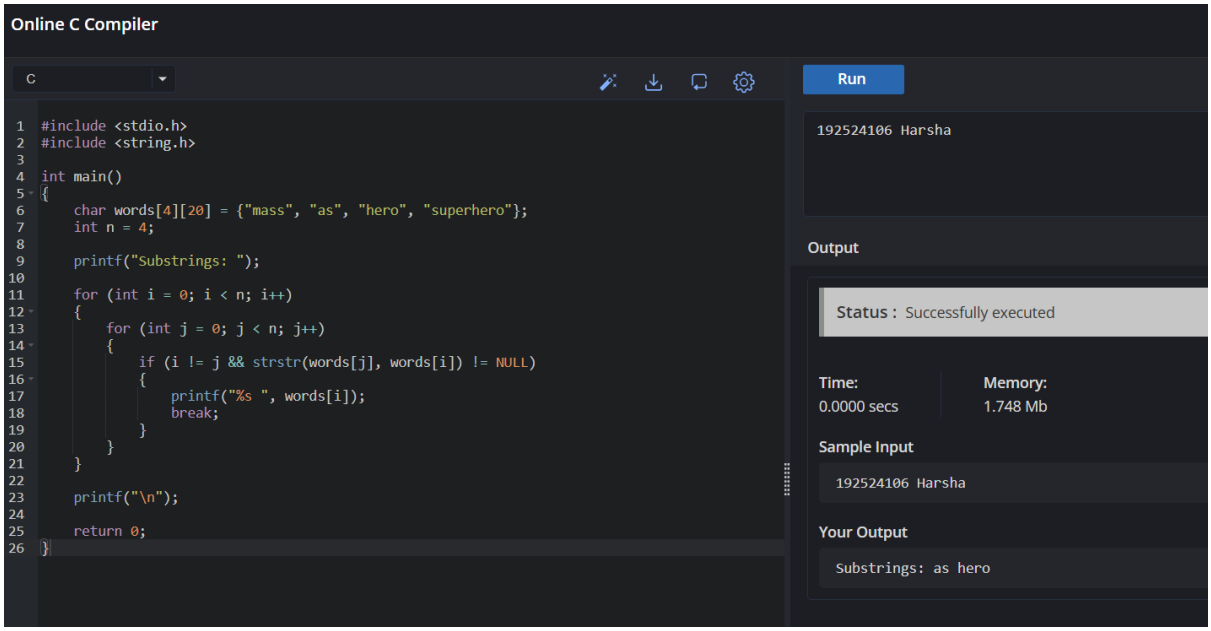
int main()
{
    char words[4][20] = {"mass", "as", "hero", "superhero"};
    int n = 4;

    printf("Substrings: ");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i != j && strstr(words[j], words[i]) != NULL)
            {
                printf("%s ", words[i]);
                break;
            }
        }
    }

    printf("\n");

    return 0;
}
```



30. Travelling Salesman Problem (TSP)

```
#include <stdio.h>

#include <math.h>


struct Point

{

   double x, y;

};


double distance(struct Point a, struct Point b)

{

   return sqrt(pow(a.x - b.x, 2) +

               pow(a.y - b.y, 2));

}


int main()

{

   struct Point p[] = {

      {1, 2}, {4, 5}, {7, 8}, {3, 1}

   };


   int n = 4;

   double minDist = INFINITY;
```

```
    int p1 = 0, p2 = 1;

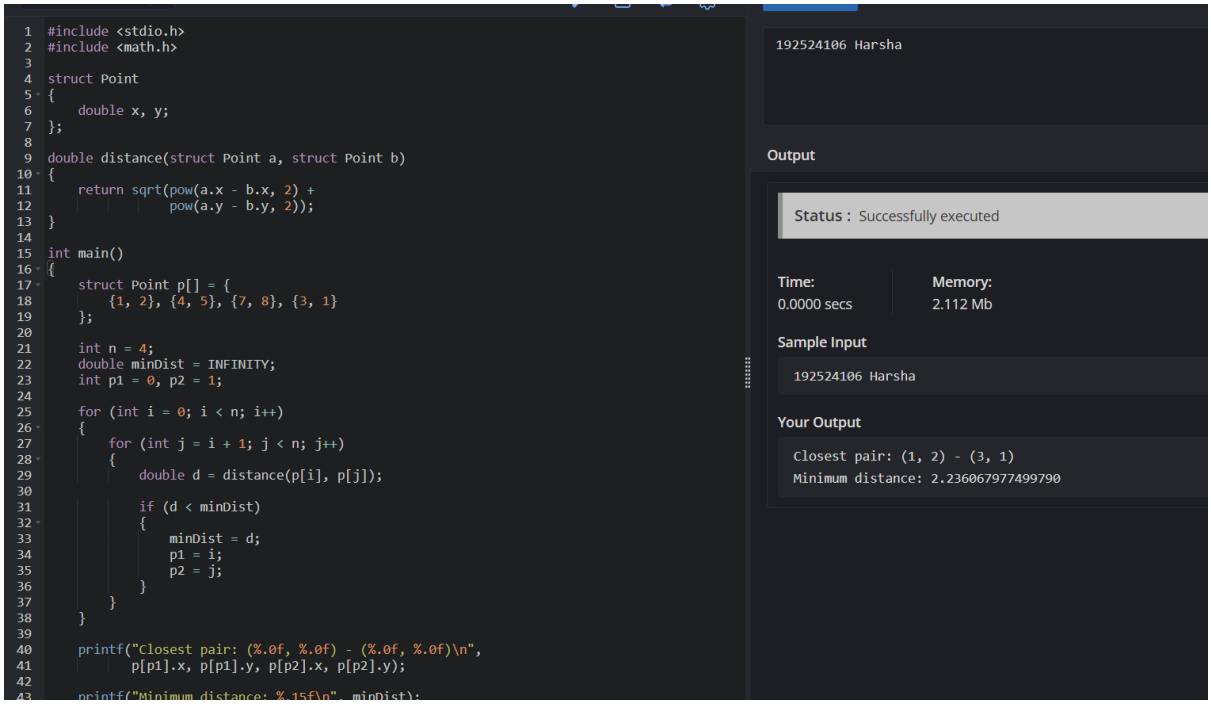
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            double d = distance(p[i], p[j]);

            if (d < minDist)
            {
                minDist = d;
                p1 = i;
                p2 = j;
            }
        }
    }

    printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n",
           p[p1].x, p[p1].y, p[p2].x, p[p2].y);

    printf("Minimum distance: %.15f\n", minDist);

    return 0;
}
```



```c
#include <stdio.h>
#include <math.h>

struct Point
{
    double x, y;
};

double distance(struct Point a, struct Point b)
{
    return sqrt(pow(a.x - b.x, 2) +
                pow(a.y - b.y, 2));
}

int main()
{
    struct Point p[] = {
        {1, 2}, {4, 5}, {7, 8}, {3, 1}
    };

    int n = 4;
    double minDist = INFINITY;
    int p1 = 0, p2 = 1;

    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            double d = distance(p[i], p[j]);

            if (d < minDist)
            {
                minDist = d;
                p1 = i;
                p2 = j;
            }
        }
    }

    printf("Closest pair: (%.0f, %.0f) - (%.0f, %.0f)\n",
           p[p1].x, p[p1].y, p[p2].x, p[p2].y);

    printf("Minimum distance: %.15f\n", minDist);
```

```
192524106 Harsha
```

Output

Status : Successfully executed

Time: 0.0000 secs

Memory: 2.112 Mb

Sample Input

```
192524106 Harsha
```

Your Output

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.236067977499790
```